

Hardware-Accelerated Reinforcement Learning Simulator for Octopus CXL Memory Pooling

Pulak Mehrotra

March 8, 2026

Abstract

The current trace-driven Octopus simulator relies on sequential, CPU-bound execution, limiting the sample efficiency of Reinforcement Learning (RL) agents such as Soft Actor-Critic (SAC)[cite: 324, 348]. To effectively train policies capable of anticipating diurnal cloud workloads [cite: 128] and transitioning to online Locally Constrained Policy Optimization (LCPO)[cite: 339], the environment must be highly parallelized. This proposal outlines the transition to a JAX-based, fully tensorized simulation architecture deployed across a 3x TITAN RTX GPU cluster, enabling millions of simulated transitions per second.

1 Phase 1: Stateless Functional Representation (JAX PyTrees)

Standard object-oriented environment states inherently bottleneck hardware acceleration. In JAX, the simulation state must be immutable and represented as a PyTree. For the target 96-host Acadia pod [cite: 538] connected to $m = 120$ Multi-Ported CXL Devices (MPDs)[cite: 538], the global state is defined without batch dimensions, allowing JAX to handle vectorization natively.

```
1 import jax.numpy as jnp
2 from flax import struct
3
4 @struct.dataclass
5 class OctopusState:
6     mpd_loads: jnp.ndarray          # Shape: (120,) - Load per MPD
7     active_vms: jnp.ndarray        # Shape: (Max_VMs, 3) - [host_id, mpd_id, size]
8     current_tick: jnp.int32        # Scalar - Time step in the 14-day trace
```

Listing 1: JAX PyTree State Definition

2 Phase 2: Tensorized Environment Dynamics

The core memory allocation logic must be rewritten as pure linear algebra. When a VM arrives at host S_i , the RL policy maps the system state to an allocation vector $a \in \mathbb{R}_{\geq 0}^m$ [cite: 31]. The graph sparsity dictates that $a_j = 0$ if the adjacency matrix $M_{ij} = 0$ [cite: 30, 33].

In the JAX environment step, this hard constraint is enforced by applying a logarithmic mask prior to the softmax normalization:

$$\tilde{w}_j = \begin{cases} w_j & \text{if } M_{ij} = 1 \\ -\infty & \text{if } M_{ij} = 0 \end{cases} \quad (1)$$

$$a_j = x \cdot \frac{\exp(\tilde{w}_j)}{\sum_{k=1}^m \exp(\tilde{w}_k)} \quad (2)$$

This operation executes via element-wise matrix multiplication, eliminating iterative masking loops and allowing constant-time execution on the GPU.

3 Phase 3: Zero-Copy Trace Execution

The production VM traces from Azure datacenters (e.g., AMS20, LON23) span approximately 14 days at a 5-minute resolution[cite: 107, 127]. To prevent PCIe bandwidth from throttling the simulation, the entire trace dataset is pre-compiled into a static three-dimensional tensor: $\mathcal{T} \in \mathbb{R}^{T \times n \times F}$, where $T = 4032$ (ticks), $n = 96$ (hosts), and F represents features like VM size and lifetime[cite: 279].

Prior to training, $\mathcal{T}$ is pinned directly to the TITAN RTX VRAM. During the environment `step()`, the `jax.lax.dynamic_slice` primitive fetches the incoming demand for `state.current_tick` entirely within silicon, achieving zero-copy data loading.

4 Phase 4: Multi-GPU Scaling Strategy (3x TITAN RTX)

To exploit the 72GB of aggregate VRAM across the 3x TITAN RTX cluster, the single-pod `step()` function is transformed using JAX’s composable parallelization primitives.

Let B_{gpu} denote the maximum number of parallel pods that fit within the 24GB VRAM of a single TITAN RTX (estimated $B_{gpu} \approx 4096$).

```

1 import jax
2
3 # 1. Intra-GPU Vectorization: Simulates B_gpu pods concurrently
4 vmapped_step = jax.vmap(step_fn, in_axes=(0, 0))
5
6 # 2. Inter-GPU Parallelization: Distributes across 3 GPUs
7 # Total parallel simulations = 3 * B_gpu
8 parallel_env_step = jax.pmap(vmapped_step, in_axes=(0, 0))

```

Listing 2: Hierarchical Parallelization

This hierarchical scaling ensures that the SAC agent experiences a massively diverse set of diurnal permutations [cite: 128, 131] and randomized link failures [cite: 519] simultaneously, stabilizing the policy gradient and closing the gap against the optimal offline baseline[cite: 524].